

Appunti di Basi di dati 1

Emanuele Romano

Indice

1	Progettazione Concettuale	1
1.1	Progettazione Concettuale con Class Diagrams	1
1.2	Ruoli nelle Associazioni	2
1.3	Navigabilità e Direzionalità	2
1.4	Tipi Enumerativi (Enumerazioni)	3
1.5	Classi di Associazione	3
1.6	Grado di un'Associazione e Relazioni Ternarie	5
1.7	Aggregazione e Composizione	6
1.8	Generalizzazione ed Ereditarietà	8
A	Bozze	10
A.1	Aspetti implementativi dell'ereditarietà	10

Progettazione Concettuale

1.1 Progettazione Concettuale con Class Diagrams

La progettazione concettuale è la fase in cui si modella il *cosa* deve essere rappresentato nella base di dati, ignorando i dettagli implementativi del DBMS. Sebbene la letteratura classica utilizzi il modello Entity-Relationship (ER), l'approccio moderno predilige l'uso dei **Class Diagrams UML**, di cui di seguito richiameremo i concetti fondamentali. Si assumerà che il lettore abbia già familiarità con i diagrammi UML in un contesto orientato agli oggetti, dunque ne faremo dei richiami piuttosto sommari e ci concentreremo sulle specificità della modellazione per basi di dati.

Il punto di partenza è un testo in linguaggio naturale (specifiche). La costruzione del diagramma segue una strategia di mapping semantico:

- **Sostantivi → Classi (Entità):** Rappresentano oggetti con esistenza autonoma (es. *Studente*, *Corso*).
- **Verbi → Associazioni (Relazioni):** Descrivono i legami logici tra le classi (es. *Iscrive*, *Sostiene*).
- **Aggettivi o Specifiche → Attributi:** Proprietà atomiche che descrivono una classe (es. *Nome*, *Matricola*).

Gli elementi costitutivi di un Class Diagram sono:

Classe: Rappresentata da un rettangolo. Nella progettazione concettuale ci si concentra su *Nome* e *Attributi*. Le *Operazioni* (metodi) vengono solitamente omesse.

Associazione: Una linea che connette due classi. Indica che esiste una correlazione logica tra le istanze delle classi coinvolte.

Cardinalità: Indicano il numero minimo e massimo di istanze di una classe che possono essere legate a una singola istanza della classe opposta.

Tabella 1.1: Sintassi delle Cardinalità UML

Sintassi	Significato
1	Esattamente uno (Obbligatorio)
0..1	Zero o uno
* o 0..*	Zero o molti
1..*	Uno o molti (Almeno uno)

Un attributo si dice **totale** se ha cardinalità minima strettamente maggiore di zero, **parziale** altrimenti: quindi il primo deve sempre avere un valore, mentre per uno parziale il valore è opzionale.

Un attributo si dice **singolo** se ha cardinalità massima uguale a uno, **multiplo** altrimenti.

Un attributo è **atomico** se il database non ne percepisce la struttura interna. Gli attributi atomici possono essere:

- **Primitivi:** Rappresentano valori elementari non scomponibili.

- *Numerici*: `int`, `float`, `decimal`.
- *Alfanumerici*: `string`, `char`.
- *Logici*: `boolean`.
- **Predefiniti Strutturati**: Tipi di dato complessi forniti dal sistema e trattati come atomici dal DBMS.
 - *Temporal*: `date`, `time`, `timestamp`.
 - *Oggetti Binari*: `blob` (immagini, file), `clob` (testi estesi).

Un errore comune è modellare come attributo ciò che dovrebbe essere una classe. La regola empirica è:

1. Se l'informazione è un valore semplice (es. *Età*), è un **Attributo**.
2. Se l'informazione ha proprietà proprie o deve essere legata ad altre entità (es. *Indirizzo* inteso come via, civico e CAP), deve essere una **Classe** autonoma.



L'insieme degli attributi deve permettere di identificare univocamente ogni istanza della classe.

Osservazione 1.1. Sebbene in Java usi `-` (`private`) e `+` (`public`), nei diagrammi per database la visibilità è meno rilevante, ma mantenere il `-` per gli attributi aiuta a ricordare l'incapsulamento dei dati.

1.2 Ruoli nelle Associazioni

Un **ruolo** identifica la funzione specifica che un'istanza di una classe svolge all'interno di un'associazione. Viene indicato alle estremità della linea di associazione.

- **Utilità**: I ruoli sono essenziali per chiarire il significato del legame, specialmente quando l'associazione è *riflessiva* (connette una classe a sé stessa) o quando esistono più associazioni tra le stesse classi.
- **Naming Convention**: Solitamente si utilizza un sostantivo che descrive la responsabilità (es. *responsabile*, *supervisore*, *partecipante*).
- **Corrispondenza OOP**: Nella programmazione ad oggetti, il nome del ruolo coincide spesso con il nome dell'attributo (o del campo) che contiene il riferimento all'oggetto correlato.

Esempio 1.1 (Associazione Riflessiva o Ricorsiva). Si consideri la classe **Persona** e l'associazione *Genitorialità*. Senza ruoli, il legame sarebbe ambiguo. Assegnando i ruoli **genitore** e **figlio** alle due estremità, la semantica del modello diventa univoca.

Osservazione 1.2. Spesso si tende a omettere il nome dell'associazione (il verbo al centro della linea) se i ruoli sono già sufficientemente esplicativi. In molti schemi professionali vengono mostrati solo i ruoli, perché sono più vicini alla struttura logica finale del database.

1.3 Navigabilità e Direzionalità

Una differenza cruciale tra la modellazione OOP e quella per basi di dati riguarda la **navigabilità** delle associazioni.

- **In Java (OOP)**: Le associazioni sono spesso *unidirezionali* per favorire il disaccoppiamento tra le classi. La direzione indica quale oggetto mantiene il riferimento (puntatore) verso l'altro.

- **Nelle Basi di Dati:** Le associazioni sono intrinsecamente *bidirezionali*. Una relazione tra due entità rappresenta un fatto logico che può essere interrogato partendo da entrambi i lati (es. tramite operatori di *Join* in SQL).

Dunque nel class diagram concettuale per database, si evitano solitamente le frecce di navigabilità. Una linea semplice indica che l'informazione è disponibile per l'interrogazione in modo simmetrico. La "direzionalità" fisica verrà stabilita solo nella fase di progettazione logica attraverso il posizionamento delle chiavi esterne.

1.4 Tipi Enumerativi (Enumerazioni)

Quando il dominio di un attributo è costituito da un insieme di valori costante, discreto e predefinito, si utilizza il costrutto della **Enumerazione**. Si tratta di un tipo di dato speciale che permette a un attributo di assumere solo uno dei valori elencati in una lista chiusa.

In UML, le enumerazioni sono rappresentate come classi con lo stereotipo «enumeration». Non hanno attributi propri, ma contengono un elenco di valori letterali che rappresentano le possibili istanze del tipo enumerativo.

1.5 Classi di Associazione

Nella modellazione concettuale, capita spesso che un'associazione tra due classi possieda delle proprietà proprie. Se un attributo non descrive intrinsecamente né la classe A né la classe B, ma esiste solo in funzione della loro relazione, ci troviamo di fronte alla necessità di una **Classe di Associazione**.

Si consideri, ad esempio, il caso classico di un sistema universitario: uno **Studente** sostiene un **Corso**. Vogliamo memorizzare il *voto* conseguito.

- Il *voto* non è un attributo dello **Studente** (egli ha voti diversi per corsi diversi).
- Il *voto* non è un attributo del **Corso** (il corso ha voti diversi per studenti diversi).

In un diagramma standard, saremmo costretti a omettere il voto o a inserirlo forzatamente in una delle due classi, creando un errore concettuale e ridondanza.

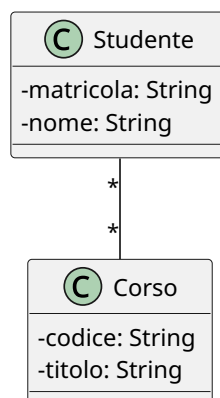


Figura 1.1: Associazione Studente-Corso.

L'UML risolve il problema permettendo a un'associazione di avere una propria classe collegata. Questa classe ha un doppio valore: è sia un legame logico tra le istanze, sia un contenitore di attributi.

Sintatticamente, si rappresenta con un rettangolo di classe collegato alla linea di associazione tramite una **linea tratteggiata**. Essa eredita implicitamente le "chiavi" di entrambe le classi coinvolte.

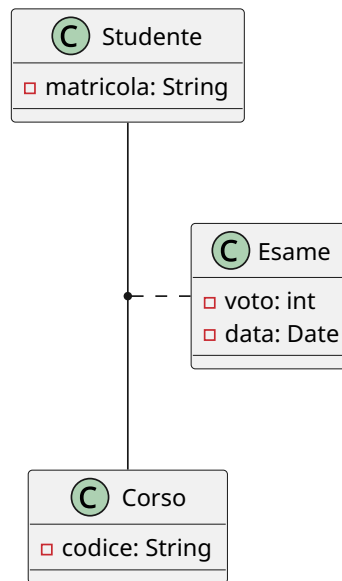


Figura 1.2: Studente-Corso con classe di associazione Esame.

In molte fasi di progettazione logica, o per chiarezza di implementazione, la classe di associazione viene "esplicitata" (o reificata). La classe di associazione diventa una classe intermedia a tutti gli effetti, e l'associazione multi-a-molti originale si trasforma in due associazioni uno-a-molti.

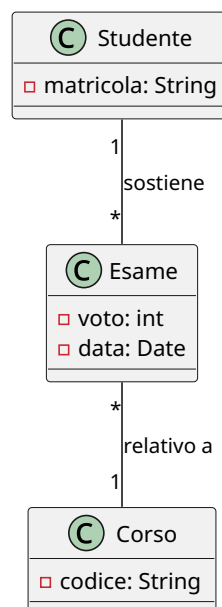


Figura 1.3: Reificazione della classe di associazione Esame.

L'esplicitazione è spesso necessaria perché molti strumenti di modellazione o linguaggi di programmazione non supportano nativamente il costrutto della classe di associazione. In questo caso, la classe intermedia (es. **Esame**) funge da "ponte" tra le due entità principali.

Tip

La classe di associazione è significativa solo su associazioni *multi-a-multi*, ma occasionalmente si usa anche per altri tipi di associazioni.

1.6 Grado di un'Associazione e Relazioni Ternarie

Il **grado** di un'associazione è definito come il numero di classi coinvolte nel legame logico.

- **Associazioni Binarie (Grado 2):** Collegano due classi. Costituiscono la quasi totalità delle relazioni in un database.
- **Associazioni N-arie (Grado $N \geq 3$):** Collegano tre o più classi contemporaneamente. Il caso più comune è l'associazione **ternaria**.

Un'associazione ternaria si rende necessaria quando il legame logico esiste solo se si considerano tre entità simultaneamente; la scomposizione in relazioni binarie separate causerebbe una perdita irreparabile di informazione semantica. In UML si rappresenta graficamente interponendo un **rombo** tra le linee che collegano le tre classi.

Si consideri, ad esempio, uno scenario di logistica industriale in cui sono coinvolte tre classi: **Fornitore**, **Prodotto** e **Progetto** (cantiere di destinazione). L'atto del rifornimento è un evento atomico: il *Fornitore A* fornisce il *Prodotto X* specificamente per il *Progetto Y*. Se provassimo a mappare questo dominio con tre relazioni binarie (*Fornitore-Prodotto*, *Prodotto-Progetto*, *Fornitore-Progetto*), sapremmo quali prodotti vende un fornitore e quali prodotti usa un progetto, ma non sapremmo mai **quale specifico fornitore ha portato quel determinato prodotto a quel singolo progetto**.

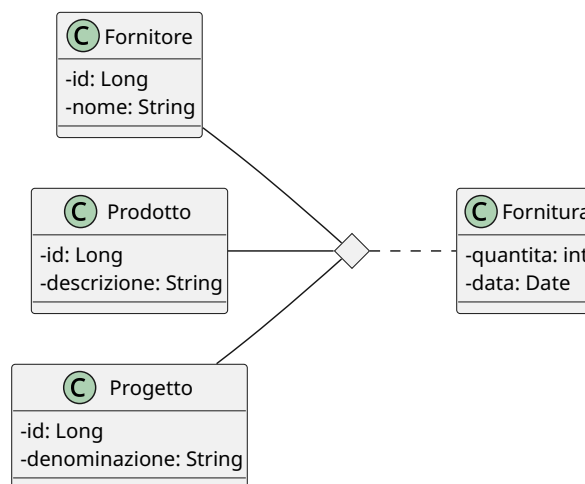


Figura 1.4: Esempio associazione ternaria con classe di associazione.

Interpretazione di una associazione n-aria

Per comprendere a fondo l'associazione ternaria, si può pensare che una delle tre classi non si colleghi singolarmente alle altre due, ma alla loro **coppia**. Nel nostro esempio, il **Progetto** non è legato separatamente a un **Prodotto** e a un **Fornitore**, ma è associato stabilmente alla coppia (*Prodotto, Fornitore*), intesa come specifica combinazione. Analogamente per le altre due classi.

Poiché i DBMS relazionali e i framework ORM (come JPA/Hibernate) non supportano le relazioni native a tre vie, si deve **esplicitare** (o reificare) l'associazione prima dell'implementazione.

La reificazione trasforma il rombo dell'associazione in una classe vera e propria, che chiameremo **AssegnazioneFornitura**. L'associazione ternaria originale viene così frantumata in tre associazioni binarie distinte di tipo **1-a-Molti**:

- Da Fornitore a AssegnazioneFornitura (Cardinalità $1 \rightarrow *$)
- Da Prodotto a AssegnazioneFornitura (Cardinalità $1 \rightarrow *$)
- Da Progetto a AssegnazioneFornitura (Cardinalità $1 \rightarrow *$)

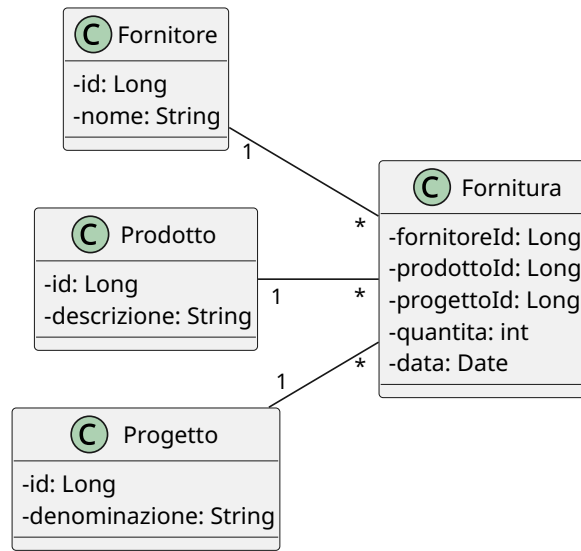


Figura 1.5: Reificazione dell'associazione ternaria.

Nella realtà, gli eventi n-ari catturano quasi sempre proprietà quantitative. Riprendendo l'esempio precedente, l'atto della fornitura genera intrinsecamente una *quantità* di pezzi consegnati e una *data* di consegna. Questi attributi non appartengono a nessuna delle tre classi isolate, ma alla loro combinazione. In UML si modella una **Classe di Associazione** (es. **Fornitura**) collegata tramite una linea tratteggiata al rombo dell'associazione ternaria.

Il processo di esplicitazione in questo scenario è immediato: la classe di associazione **Fornitura** viene promossa a classe centrale del cluster, inglobando gli attributi **quantità** e **data**. Le tre classi originarie si collegano a essa tramite relazioni binarie dirette.

⚠ La trappola delle cardinalità ternarie

Nelle associazioni ternarie UML, la cardinalità posta vicino a una classe (es. **Progetto**) indica quante istanze di quella classe possono combinarsi con una **coppia fissa** di istanze delle altre due classi (**Fornitore** e **Prodotto**). Questa logica è controintuitiva e differisce dai diagrammi ER classici (notazione di Chen), inducendo spesso i progettisti in errore. Sostituire subito la ternaria con una classe esplicitata azzerà ogni ambiguità interpretativa.

1.7 Aggregazione e Composizione

Nello sviluppo dei Class Diagrams, le associazioni standard indicano un legame logico simmetrico. Tuttavia, per modellare scenari in cui esiste un rapporto di possesso o

di subordinazione strutturale, l'UML introduce le **relazioni tutto-parte** (part-whole relationships). Queste si dividono in due varianti fondamentali, distinte in base al vincolo sul ciclo di vita degli oggetti coinvolti: l'aggregazione e la composizione.

L'**aggregazione** rappresenta una relazione in cui un oggetto “Tutto” (aggregante) contiene o colleziona uno o più oggetti “Parte” (aggregati), ma questi ultimi mantengono un'esistenza autonoma all'interno del dominio. Si tratta di un legame “*has-a*” ad accoppiamento debole.

- **Sintassi UML:** Si rappresenta graficamente con una linea che termina con un **rombo vuoto** (bianco) posizionato sul lato della classe “Tutto”.
- **Ciclo di vita:** La distruzione dell'oggetto aggregante **non** comporta la distruzione degli oggetti aggregati.

Esempio 1.2 (Università e Professore). Si consideri la classe **Università** (il Tutto) e la classe **Professore** (la Parte). Un'università raccoglie un insieme di docenti. Tuttavia, se l'università dovesse cessare la sua attività, i singoli professori non verrebbero eliminati dal sistema: essi continuerebbero ad esistere come entità autonome e potrebbero essere associati ad altri atenei.

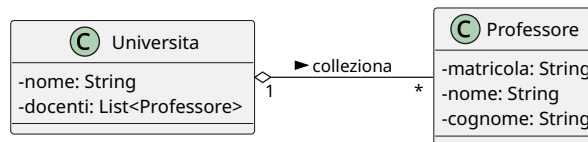


Figura 1.6: Esempio di aggregazione

La **composizione** è una forma di aggregazione più stringente, esclusiva e gerarchica. In questo scenario, l'oggetto “Parte” è una componente indissolubile del “Tutto”. Una parte può appartenere a un solo tutto alla volta e non ha alcuna autonomia esistenziale.

- **Sintassi UML:** Si rappresenta con una linea che termina con un **rombo pieno** (nero) posizionato sul lato della classe “Tutto”.
- **Ciclo di vita:** Il ciclo di vita delle parti è totalmente subordinato a quello del tutto. Di conseguenza, la distruzione dell'oggetto composto (Tutto) determina l'istanza di **distruzione automatica** di tutte le sue parti. La cardinalità dal lato del tutto è rigorosamente 1 o 0..1.

Esempio 1.3 (Ordine ed Elemento d'Ordine). Si consideri un sistema di e-commerce con le classi **Ordine** (il Tutto) e **RigaOrdine** (la Parte, che descrive il prodotto e la quantità). Una riga d'ordine ha significato logico solo se contestualizzata all'interno dello specifico acquisto. Se l'utente decide di eliminare l'intero **Ordine**, il sistema deve rimuovere a cascata tutte le relative istanze di **RigaOrdine**.

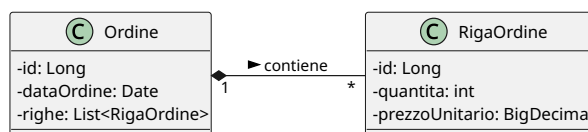


Figura 1.7: Esempio di composizione

Q L'aggregazione non ha alcuna necessità tecnica stringente: una semplice associazione fa esattamente lo stesso identico lavoro. A livello di tabelle, sia un'associazione generica sia un'aggregazione si tradurranno nello stesso modo, la differenza è puramente semantica e documentale, non implementativa. Alcuni autorevoli ingegneri del software (tra cui Martin Fowler, uno dei padri dell'architettura software moderna) sostengono da anni che l'aggregazione UML sia fundamentalmente inutile perché la linea di demarcazione con l'associazione standard è troppo sfumata e crea più ambiguità che benefici. Fowler la definisce ironicamente "un effetto placebo grafico".

Il caso della composizione è invece più significativo, perché il diagramma fornisce un'informazione in più: la parte non può esistere senza il tutto.

1.8 Generalizzazione ed Ereditarietà

La **generalizzazione** (o ereditarietà) modella una relazione di tipo “IS-A” (è-un) tra una classe più generale, detta **Superclasse** (o classe padre), e una o più classi più specifiche, dette **Subclassi** (o classi figlie).

Le subclassi ereditano automaticamente tutti gli attributi, i ruoli e le associazioni della superclasse, senza necessità di ridefinirli, e possono estendere il modello introducendo attributi o associazioni propri.

Sintassi UML: Si rappresenta graficamente con una linea che termina con una **freccia con la punta triangolare vuota** rivolta verso la superclasse.

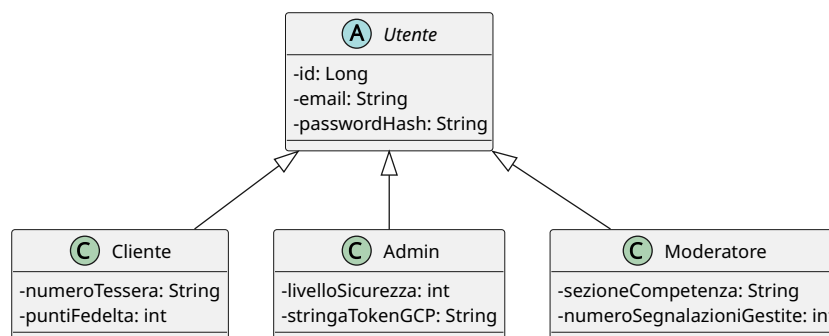


Figura 1.8: Esempio di ereditarietà

È fondamentale comprendere che **l'ereditarietà non esiste nel modello relazionale delle basi di dati**. I database relazionali non possiedono strutture gerarchiche native; essi conoscono esclusivamente il concetto di tabella (relazione), che è per sua natura una struttura “piatta” e bidimensionale.

La generalizzazione appartiene quindi in modo esclusivo al livello della **progettazione concettuale**. In questa fase, lo scopo del Class Diagram è modellare il dominio reale nel modo più fedele e naturale possibile, catturandone la semantica e i vincoli logici (ovvero i rapporti di sotto-tipo/sopra-tipo) ed eliminando qualsiasi vincolo tecnologico. Il problema di come “appiattire” questa gerarchia per adattarla a un database relazionale sarà l'obiettivo centrale della successiva fase di progettazione logica.

Una generalizzazione può essere classificata combinando due tipi di vincoli ortogonali, utili a descrivere la natura delle istanze nel dominio reale:

1. Completezza (Totale vs Parziale):

- **Totale (Completa):** Ogni istanza della superclasse deve obbligatoriamente appartenere ad almeno una delle subclassi. Non esistono elementi “generici” (corrisponde al concetto di *Classe Astratta* in OOP).
 - **Parziale (Incompleta):** Un’istanza della superclasse può esistere senza dover per forza appartenere a una delle subclassi catalogate.
2. **Disgiunzione (Disgiunta vs Sovrapposta):**
- **Disgiunta:** Un’istanza della superclasse può appartenere al massimo a **una sola** delle subclassi. Le sottoclassi sono mutuamente esclusive.
 - **Sovrapposta (Overlapping):** Una singola istanza della superclasse può appartenere contemporaneamente a più subclassi distinte.

Ci sono dunque quattro combinazioni possibili. Nel diagramma UML, i vincoli della generalizzazione si indicano inserendo un’etichetta testuale racchiusa tra parentesi graffe (*Generalization Set*) in prossimità delle linee di ereditarietà, specificando la coppia di proprietà (es: {complete, overlapping}).

Esempio 1.4 (Esempi di Vincoli). Si considerino i seguenti scenari aziendali:

- **Totale / Disgiunta:** La superclasse **ContoBancario** ha come subclassi **ContoCorrente** e **ContoDeposito**. Un conto deve essere obbligatoriamente di un tipo o dell’altro, e non può essere entrambe le cose contemporaneamente.
- **Parziale / Sovrapposta:** La superclasse **Persona** ha come subclassi **Studente** e **Dipendente**. All’interno dell’anagrafica universitaria, una persona può essere solo un cittadino esterno (parziale), oppure può essere contemporaneamente uno studente che lavora per l’ateneo (sovrapposta).

Bozze

A.1 Aspetti implementativi dell'ereditarietà

Poiché il modello relazionale delle basi di dati si basa su tabelle bidimensionali e non prevede il concetto nativo di ereditarietà, lo sviluppatore backend deve scegliere come tradurre la gerarchia UML in tabelle SQL. Esistono tre strategie standard, ognuna con precisi trade-off ingegneristici.

Immaginiamo la gerarchia: **Utente** (id, email) → **Cliente** (voti) e **Admin** (livelloAccesso).

1. Tabella Unica (Single Table Inheritance)

La gerarchia viene collassata in **un'unica grande tabella** che contiene tutti gli attributi della superclasse e delle subclassi. Viene aggiunta una colonna speciale detta **Discriminator** (es. `tipo_utente`) per capire quale riga rappresenti quale classe.

- **Vantaggi:** Performance massime. Le query non richiedono alcuna operazione di JOIN per recuperare i dati specifici.
- **Svantaggi:** Violazione della normalizzazione. Gli attributi specifici delle subclassi (es. `voti` per l'admin o `livelloAccesso` per il cliente) devono obbligatoriamente accettare valori NULL. Se le subclassi hanno molti attributi unici, la tabella diventa estremamente sparsa e spreca spazio.

2. Tabella per Classe Concreta (Table Per Class Inheritance)

Viene creata una tabella indipendente per ogni sottoclasse concreta. Gli attributi della superclasse vengono **duplicati** in ciascuna tabella figlia. Non esiste una tabella per la superclasse.

- **Vantaggi:** Struttura pulita se si interrogano sempre e solo le classi specifiche (es. cerco solo i clienti). Nessun valore NULL forzato.
- **Svantaggi:** Le query polimorfiche (es. "Seleziona tutti gli utenti del sistema, indipendentemente dal tipo") sono un incubo computazionale: richiedono pesanti operazioni di UNION tra tutte le tabelle. Inoltre, non è possibile definire una Foreign Key che punti genericamente a un **Utente**.

3. Tabelle Congiunte (Joined / Table Per Subclass Inheritance)

Viene creata una tabella per la superclasse e una tabella distinta per ciascuna subclassi. Le tabelle delle subclassi contengono **solo** i propri attributi specifici e una chiave primaria che funge contemporaneamente da **Chiave Esterna (FK)** verso la tabella padre.

- **Vantaggi:** Massimo rigore relazionale e pulizia matematica (rispetta la normalizzazione). Nessun valore NULL inutile, nessuna duplicazione di colonne.
- **Svantaggi:** Calo drastico delle performance su gerarchie profonde. Ogni singola lettura di una sottoclasse richiede un'operazione di JOIN tra la tabella padre e la tabella figlia.